AI Pioneers

CCA-F | Module 2

Lab 2.2

Connecting the Ecosystem: MCP Servers & Built-in Claude Code Tools

---

Scenario: Giving Claude Code Real Context at an Outdoor-Gear Store

Module	M2 — Tooling & Context for Coding Agents
Lab type	Hands-on · Self-paced or Instructor-Led · Run inside Claude Code
Duration	~40 minutes (hands-on)
Environment	Blue Labs VM · Claude Code · Python 3.10+ (for the local MCP servers)
Sections	S4 (MCP server integration), S5 (Built-in Tool Usage)

Lab Objectives

By the end of this lab you will be able to:

- **Connect more than one MCP server** — declare an orders source and a docs source in `.mcp.json` so Claude Code can pull from several independent sources in one session, then answer a question that needs both at once.
- **Drive the built-in tools deliberately** — use `Glob` to find files, `Grep` to locate exact usages, `Read` for just the file that matters, `Edit` to change an existing file in place, and `Write` to create a new one.
- **Explore incrementally** — practise the find → read-only-what-matters → change loop, and see why it beats loading the whole repo "just in case."

1. Overview

An agent is only as good as the context it can reach, and how precisely it can act on it. This lab gives Claude Code real context to work with and then asks it to change a codebase in the efficient way. First you wire up two MCP servers so the agent can pull from multiple sources without copying data. Then you use its built-in tools — Glob, Grep, Read, Edit — to explore and refactor a small TypeScript project a little at a time instead of all at once. The whole lab is done inside Claude Code, run from the project folder, and continues the NorthPeak Outfitters scenario from Lab 2.1.

SN	Section	What you will do	Core idea
Ex 1	**S4 — MCP Servers**	Connect an orders source and a docs source via .mcp.json, then answer a question that needs both.	More sources, declared once; the agent calls them instead of you pasting data.
Ex 2	**S5 — Built-in Tools**	Glob the tests, Grep a deprecated call, Read the signature, Edit the fixes, then Write a new file.	Precise, scriptable actions beat "look at everything."
Ex 3	**S5 — Incremental Exploration**	Rename one analytics event using Grep → Read → Edit, touching exactly one file.	Locate precisely, read narrowly, change minimally.

2. Scenario

You are working on the internal tooling for NorthPeak Outfitters, the same fictional outdoor-gear store from Lab 2.1. Two kinds of context live in different places: live order data (status, items, tracking) sits in an orders service, and customer-facing policy (shipping, returns, warranty) lives as a set of documents. A support engineer using Claude Code needs both at once — "this order was delivered; can the customer still return an item, and what condition rules apply?" needs the order AND the returns policy.

Separately, the engineering team has a small TypeScript codebase with a deprecated analytics function that needs migrating, and an event name that needs a one-letter rename. The point is not to read the whole repo — it is to find the exact call sites and change only those.

Your task is to set Claude Code up with the right context and then drive it precisely. You will:

- Connect two MCP servers (orders + docs) and ask a question that forces the agent to combine both sources.
- Migrate a deprecated function across the codebase using Glob, Grep, Read, and Edit — not by dumping the project into context.
- Make a one-line rename by locating the single call site first, reading only that file, and editing just that line.

The two MCP sources (declared in .mcp.json)

```
northpeak-orders   →  get_order(order_id)   find_orders_by_email(email)
northpeak-docs     →  list_docs()   read_doc(name)   search_docs(query)
Verify inside Claude Code with  /mcp  — both should show as connected.
```

Why these three ideas together?

MCP servers give the agent data instead of guesses; the built-in tools give it precise ways to act on a codebase; and incremental exploration keeps it fast, cheap, and accurate. Wiring up sources but then reading the whole repo

wastes the precision the tools give you; using the tools well but starving the agent of sources makes it invent answers. The three reinforce each other — good context, precise actions, minimal footprint.

3. Pre-requisites

3.1 Skilljar Videos — Complete Before the Session

Course	Lessons to watch	Covers
Claude Code in Action	MCP servers with Claude Code (S4); Adding context; Making changes (S5)	S4, S5
Model Context Protocol: Advanced Topics	The STDIO transport; The StreamableHTTP transport (supporting)	S4
Claude Code 101	MCP (S4); How Claude Code works (supporting) (S5)	S4, S5

The lab assumes you have Claude Code installed and have completed Lab 2.1, so you understand tools and the tool-use loop. The instructor will not re-teach Claude Code basics — bring questions to the Doubt Session instead.

3.2 Environment Check

Run these checks before the session starts. If any step fails, contact the instructor or check the Blue Labs SOP.

- Start your Blue Labs VM: open https://www.bluelabs.studio/, locate your VM, click Start, then Connect.
- Confirm Claude Code is installed:

```
claude --version
```

- Create the project folder and enter it:

```
mkdir lab-2.2-mcp-and-tools
cd lab-2.2-mcp-and-tools
```

- Create the lab resource files. There is no pre-packaged bundle: build every file listed in Appendix A (Section 7) at the exact path shown — the .mcp.json, the two MCP servers, the data files, and the sample_codebase. Do this before the next step, since it creates requirements.txt.
- Create a virtualenv and install the MCP server dependency:

```
python -m venv .venv && source .venv/bin/activate
# Windows: .venv\Scripts\activate
pip install -r requirements.txt          # mcp>=1.2.0
```

- Start Claude Code FROM this folder so it reads .mcp.json, then verify the servers:

```
claude
# then inside Claude Code:
```

```
/mcp
```

Interpreter note & ANTHROPIC_API_KEY

On the Blue Labs VM your API key is pre-configured — you do not set it manually. The one thing that trips people up is the interpreter: `.mcp.json` launches the servers with `python3`, which must have the `mcp` package installed. If a server fails to connect, point the command at your venv interpreter — e.g. `".venv/bin/python"` (Windows: `".venv\\Scripts\\python.exe"`) — and re-run /mcp.

4. Exercises

Do these inside Claude Code, started from the project folder. Each is short. The prompts below are what to type at Claude Code; each step shows the expected result and a "What good looks like" box so you can self-check.

Exercise 1: Wire up and use two MCP servers (S4) ~12 min

Background

An MCP server is an external source Claude Code can call. The wiring is already done for you in `.mcp.json`, which declares two servers: `northpeak-orders` (order lookups from `data/orders.json`) and `northpeak-docs` (policy documents from `data/docs/`). Declaring more than one gives the agent several independent sources in the same session — without you copy-pasting any data.

The config Claude Code reads on startup:

```
{
  "mcpServers": {
    "northpeak-orders": {
      "command": "python3",
      "args": ["mcp_servers/orders_server.py"]
    },
    "northpeak-docs": {
      "command": "python3",
      "args": ["mcp_servers/docs_server.py"]
    }
  }
}
```

Task

Confirm both servers are connected, warm up with a single-source question, then ask one question that forces Claude Code to combine both sources — the order state from one server and the policy rules from the other — in a single answer.

Step 1 — Confirm both servers are connected

Start Claude Code from the lab folder, then run `/mcp`. You should see both servers listed with their tools: `get_order`, `find_orders_by_email`, `list_docs`, `read_doc`, `search_docs`.

```
/mcp
```

Step 2 — Warm up with a single-source question

Ask a question only the orders server can answer. Claude Code should call `northpeak-orders.get_order` rather than guessing:

```
What's the status of order NP-100245?
```

Expected: a `get_order("NP-100245")` call returning status `shipped`, items `["TENT-2P-RX", "BAG-20F-DN"]`, with a tracking number.

Step 3 — Ask a multi-source question

Now ask something that needs both servers at once:

```
Order NP-100190 was delivered. The customer wants to return one item —
are they still inside the return window, and what condition rules apply?
```

Expected tool sequence: `northpeak-orders.get_order("NP-100190")` (status `delivered`, items `["BOOT-GTX-M", "FILT-PMP"]`), then `northpeak-docs.read_doc("returns-policy")` (or `search_docs("return")`), then a combined answer.

What good looks like

The answer cites the delivered status from the orders server AND the 30-day window plus condition rules from the returns doc — and it flags that the order contains boots (BOOT-GTX-M), so worn footwear cannot be returned. Neither fact was pasted in; each came from its own MCP source.

Try this too: ask *"Which policy docs do you have?"* and watch Claude Code call `list_docs` (returning `returns-policy`, `shipping-policy`, `warranty`) instead of guessing.

Reflection Questions

- You asked one question that needed an order's status and a policy rule. Why is declaring two MCP servers better than pasting the order JSON and the returns doc into the chat yourself — what do you gain across a long session?
- The agent chose get_order for the order and read_doc for the policy. What in each tool's name and description makes that routing obvious — and how does this connect to the "strong tool interface" idea from Lab 2.1?
- The returns answer depended on the order containing boots. If the agent had only the returns doc (no orders server), what would it have to do — and where would that go wrong?

Exercise 2: A precise refactor with the built-in tools (S5) ~15 min

Background

Glob (find files by pattern), Grep (search file contents), Read, and Edit are precise, scriptable actions on a codebase — reach for the right one instead of asking the model to "look at everything." In `sample_codebase/`, `logEvent(name, payload)` in `src/analytics.ts` is deprecated; new code should call `track({ name, props })` instead. A few call sites still use the old function.

Task

Migrate the deprecated calls by locating them precisely with Glob and Grep, reading only the file that defines the replacement, and editing one file at a time — not by reading the whole project.

Step 1 — Glob the test files

Map the shape of the project first:

```
Glob for all *.test.ts files under sample_codebase and list them.
```

Expected: `sample_codebase/tests/notifications.test.ts` and `sample_codebase/tests/orders.test.ts`.

Step 2 — Grep for the deprecated call

Find every place the old function is actually called:

```
Grep for `logEvent(` in sample_codebase/src and show the matches.
```

Expected: four call sites — two in `notifications.ts` and two in `orders.ts`. Grep also matches the deprecated definition and its comment in analytics.ts, so you will see six lines in total — the four calls are the ones to migrate.

Step 3 — Read the replacement's signature

Open only the file that defines the new function, so you migrate correctly:

```
Read sample_codebase/src/analytics.ts so we know the track() signature.
```

The contract is `track({ name, props })` — where `logEvent(name, payload)` took two positional arguments, `track` takes a single object.

Step 4 — Edit one file first

Migrate the calls in one file, and update its import:

```
In sample_codebase/src/notifications.ts, replace each logEvent(name, payload)
call with track({ name, props }), and update the import from logEvent to track.
```

The migrated file:

```
import { track } from "./analytics";

export function sendOrderShipped(orderId: string, email: string): void {
  // ... send the "your order shipped" email ...
  track({ name: "order_shipped_email", props: { orderId, email } });
}

export function sendReturnApproved(orderId: string): void {
  // ... send the "return approved" email ...
  track({ name: "return_approved_email", props: { orderId } });
}
```

Step 5 — Stretch: migrate src/orders.ts too

Repeat the Edit for the two call sites in orders.ts:

```
import { track } from "./analytics";
```

```
export function markDelivered(orderId: string): void {
  track({ name: "order_delivered", props: { orderId } });
}

export function cancelOrder(orderId: string, reason: string): void {
  track({ name: "order_cancelled", props: { orderId, reason } });
}
```

Verify by Grepping `logEvent(` in `src/` again — it should now match only the deprecated definition and its comment in `analytics.ts`, with no live calls left.

Step 6 — Write: record the migration in a new file

Edit changes an existing file in place; `Write` creates a brand-new one (or overwrites a whole file). Use Write to drop a short migration note next to the code so the change is documented:

```
Write a new file sample_codebase/MIGRATION.md that records the
logEvent → track migration, and notes the upcoming order_cancelled → order_canceled
rename.
```

The new file Claude Code should create:

```
# Migration Notes

- Replaced deprecated logEvent(name, payload) with track({ name, props })
  across src/notifications.ts and src/orders.ts; imports updated.
- Next (Exercise 3): rename analytics event order_cancelled -> order_canceled (one
L).
```

What good looks like

Claude Code uses Glob/Grep to locate work precisely, Reads only analytics.ts for the signature, Edits so every call uses track({ name: "...", props: { ... } }) with the import updated, and uses Write — not Edit — to create the new MIGRATION.md. It never loads the whole repo “just in case.”

Reflection Questions

- You used Glob, then Grep, then Read, then Edit — in that order. What would you have lost by skipping straight to “read every file in sample_codebase” before changing anything?
- The migration changed both the call (`logEvent(…)` → `track(…)`) and the import. Why is updating the import part of the same minimal edit, and what breaks if you change the call but forget the import?
- Grep found four call sites across two files. How does Grep-before-Edit change your confidence that you have migrated everything, compared with reading files top to bottom looking for calls?
- You used `Edit` to change existing files and `Write` to create MIGRATION.md. Why is reaching for Write (whole new file) the wrong tool for a two-line change inside an existing file — and Edit the wrong tool for a file that does not exist yet?

Exercise 3: Explore incrementally, not all at once (S5) ~10 min

Background

The find → read-only-what-matters → change loop is what keeps an agent fast, cheap, and accurate on a real codebase. Loading everything "just in case" is the anti-pattern. Here you rename one analytics event — `order_cancelled` → `order_canceled` (one L) — and the efficient path touches exactly one file.

Task

Make the rename by locating the single call site with Grep, reading only that file, and editing just that line. Then contrast the cost of doing it the heavy way.

Step 1 — Ask for the change

```
We're renaming the analytics event `order_cancelled` to
`order_canceled` (one L). Make that change.
```

Step 2 — Watch the efficient path

The right approach is `Grep` for `order_cancelled` (one hit, in `src/orders.ts`), `Read` only that file, then `Edit` just that line:

```
// before
track({ name: "order_cancelled", props: { orderId, reason } });
// after
track({ name: "order_canceled", props: { orderId, reason } });
```

(If you did the Exercise 2 stretch, the call is already `track(…)`; if not, it is still `logEvent(…)` — either way only the event name changes.)

Step 3 — Contrast the heavy way

Try instead: *"read every file in sample_codebase first."* Notice how much more context that burns for the very same one-line change — that is the cost the incremental loop avoids.

What good looks like

The rename touches exactly one file (src/orders.ts), found by Grep, with no unnecessary reading of unrelated files. Locate precisely, read narrowly, change minimally. Finally, update MIGRATION.md so the "Next (Exercise 3)" line records the rename as done.

Reflection Questions

- A one-letter rename and a whole-repo read produce the same final diff. For a real monorepo, why does the path you take to that diff matter as much as the diff itself?
- Grep located the event name in one place. When would "read the whole file" (or several files) genuinely be the right call, and how do you tell that case apart from this one?
- Across all three exercises you gave the agent good sources (MCP) and then acted with precise tools. How do those two halves reinforce each other — what goes wrong if you have one without the other?

5. Debrief & Reflection

5.1 Self-Check Before You Leave

Answer each question from memory first, then check yourself against the exercises above.

#	Question
1	Where do you declare the MCP servers Claude Code should launch, and how do you confirm inside Claude Code that they connected?
2	You need an order's status and a policy rule in one answer. How many MCP sources are involved, and why is that better than pasting the data yourself?
3	Name the four built-in tools used in this lab and the one-line job of each.
4	You must migrate a deprecated function across a codebase. In what order do you reach for Glob, Grep, Read, and Edit — and why that order?
5	For a one-line rename, what is the efficient loop, and what does “read everything first” cost you for the same result?

5.2 Common Mistakes

Mistake	Why it matters and what to do instead
Pasting data into the chat instead of using a server.	Copy-pasted order JSON or policy text goes stale and bloats context. Declare the source in .mcp.json and let the agent call get_order / read_doc.
Server shows as not connected after /mcp.	Almost always the interpreter: the python3 in .mcp.json must have the mcp package. Point the command at your venv interpreter and re-run /mcp.
Asking the model to “look at everything.”	Dumping the repo wastes context and hides the signal. Glob to find, Grep to locate, Read only what matters, Edit minimally.
Editing the call but not the import.	Changing logEvent(…) to track(…) without updating the import leaves the file broken. The minimal edit includes the import line.
Reading whole files to find a symbol.	Grep finds every call site deterministically and cheaply; eyeballing files misses some and costs more. Grep first, then read narrowly.

6. Quick Reference

6.1 .mcp.json — Two Servers

```
{
  "mcpServers": {
```

```
    "northpeak-orders": {
      "command": "python3",
      "args": ["mcp_servers/orders_server.py"]
    },
    "northpeak-docs": {
      "command": "python3",
      "args": ["mcp_servers/docs_server.py"]
    }
  }
}
# venv note: swap "python3" for ".venv/bin/python"
# (Windows: ".venv\\Scripts\\python.exe") if a server won't connect.
```

6.2 MCP Tools Exposed by the Servers

Server	Tool	What it returns
northpeak-orders	`get_order`	One order by ID (status, items, tracking, email).
northpeak-orders	`find_orders_by_email`	All orders for a customer email.
northpeak-docs	`list_docs`	Names of all policy documents.
northpeak-docs	`read_doc`	Full text of one policy doc by name.
northpeak-docs	`search_docs`	Docs containing a query, with a snippet.

6.3 Built-in Tools — When to Reach for Each

Tool	Use it to
`Glob`	Find files by pattern (e.g. all *.test.ts) to map the project.
`Grep`	Search file contents for a symbol or string to locate exact call sites.
`Read`	Open only the file that matters (e.g. the one defining a signature).
`Edit`	Change part of an existing file — a minimal, targeted edit.
`Write`	Create a new file (or overwrite one wholesale). Use for new files, not in-place tweaks.

6.4 The Refactor — Before → After

```
// deprecated                          // migrated
logEvent("order_shipped_email",   ->   track({ name: "order_shipped_email",
         { orderId, email });                  props: { orderId, email } });

// one-letter rename (Exercise 3)
track({ name: "order_cancelled", ... })  ->  track({ name: "order_canceled", ... })
```

6.5 File Inventory

File	Section	Purpose
`.mcp.json`	S4	Declares the two MCP servers Claude Code launches.
`mcp_servers/orders_server.py`	S4	Orders source: get_order, find_orders_by_email.
`mcp_servers/docs_server.py`	S4	Docs source: list_docs, read_doc, search_docs.
`data/orders.json, data/docs/`	S5	Order records and policy documents.
`sample_codebase/`	S5	TypeScript project for the refactor exercises.
`(this handout)`	all	Tasks, prompts and expected results are inline in this handout (Sections 4–5); no separate answer file is needed.

6.6 Further Reading

- Claude Code — overview: https://docs.claude.com/en/docs/claude-code/overview
- Model Context Protocol: https://modelcontextprotocol.io
- Claude Code in Action on Skilljar — MCP servers with Claude Code (S4); Claude Code 101 on Skilljar — MCP (S4)
- Claude Code in Action on Skilljar — Adding context; Making changes (S5); Claude Code 101 on Skilljar — How Claude Code works (supporting) (S5)

7. Appendix A — Lab Resource Files (Create These First)

These are the files the lab operates on. In this version of the lab there is no pre-packaged bundle — you create them yourself. Make a folder named lab-2.2-mcp-and-tools and, inside it, create each file below at the exact path shown, with the exact contents given. You can use any editor, or paste a block and ask Claude Code to write the file. The sample_codebase files are the starting (pre-migration) state; you deliberately change them in Exercises 2 and 3.

Folder layout

```
.
├─ .mcp.json
├─ requirements.txt
├─ mcp_servers/
│   ├─ orders_server.py
│   └─ docs_server.py
├─ data/
│   ├─ orders.json
│   └─ docs/
│      ├─ returns-policy.md
│      ├─ shipping-policy.md
│      └─ warranty.md
└─ sample_codebase/
    ├─ src/
    │   ├─ analytics.ts
    │   ├─ notifications.ts
```

```
   │  └─ orders.ts
   └─ tests/
      ├─ notifications.test.ts
      └─ orders.test.ts
```

Appendix A — Config & data

File: .mcp.json

```
{
  "mcpServers": {
    "northpeak-orders": {
      "command": "python3",
      "args": ["mcp_servers/orders_server.py"]
    },
    "northpeak-docs": {
      "command": "python3",
      "args": ["mcp_servers/docs_server.py"]
    }
  }
}
```

File: requirements.txt

```
mcp>=1.2.0
```

File: mcp_servers/orders_server.py

```
"""NorthPeak orders MCP server.

Exposes two tools over stdio:
  - get_order(order_id): look up a single order by its ID.
  - find_orders_by_email(email): list all orders for a customer email.

Data source: data/orders.json (resolved relative to the project root, i.e.
the parent of this file's directory), so the server works no matter which
folder Claude Code is started from.
"""

import json
from pathlib import Path

from mcp.server.fastmcp import FastMCP

mcp = FastMCP("northpeak-orders")

# data/orders.json lives at <project root>/data/orders.json
ORDERS_PATH = Path(__file__).resolve().parent.parent / "data" / "orders.json"


def _load_orders() -> dict:
    with open(ORDERS_PATH, "r", encoding="utf-8") as f:
        return json.load(f)


@mcp.tool()
def get_order(order_id: str) -> dict:
    """Return a single order by its ID (e.g. "NP-100245").
```

```
    The result includes status, items, tracking number, and customer email.
    Returns an error object if the order ID is not found.
    """
    orders = _load_orders()
    order = orders.get(order_id)
    if order is None:
        return {"error": f"No order found with id {order_id!r}"}
    return order


@mcp.tool()
def find_orders_by_email(email: str) -> list:
    """Return every order placed by the given customer email address.

    Matching is case-insensitive. Returns an empty list if none match.
    """
    orders = _load_orders()
    email_norm = email.strip().lower()
    return [o for o in orders.values() if o.get("email", "").lower() == email_norm]


if __name__ == "__main__":
    mcp.run()
```

File: mcp_servers/docs_server.py

```
"""NorthPeak policy-docs MCP server.

Exposes three tools over stdio:
  - list_docs(): names of all policy documents.
  - read_doc(name): full text of one policy doc by name.
  - search_docs(query): docs whose text contains the query, with a snippet.

Data source: data/docs/*.md (resolved relative to the project root, i.e.
the parent of this file's directory). A doc's "name" is its filename without
the .md extension, e.g. "returns-policy".
"""

from pathlib import Path

from mcp.server.fastmcp import FastMCP

mcp = FastMCP("northpeak-docs")

DOCS_DIR = Path(__file__).resolve().parent.parent / "data" / "docs"


def _doc_paths() -> list:
    return sorted(DOCS_DIR.glob("*.md"))


@mcp.tool()
def list_docs() -> list:
    """Return the names of all policy documents (without the .md extension)."""
    return [p.stem for p in _doc_paths()]


@mcp.tool()
```

```
def read_doc(name: str) -> str:
    """Return the full text of one policy doc by name, e.g. "returns-policy".

    The name may be given with or without the .md extension.
    """
    stem = name[:-3] if name.endswith(".md") else name
    path = DOCS_DIR / f"{stem}.md"
    if not path.exists():
        available = ", ".join(p.stem for p in _doc_paths())
        return f"No doc named {stem!r}. Available docs: {available}."
    return path.read_text(encoding="utf-8")


@mcp.tool()
def search_docs(query: str) -> list:
    """Return docs whose text contains the query (case-insensitive).

    Each match is {"name", "snippet"}, where snippet is a short excerpt
    around the first occurrence of the query.
    """
    q = query.strip().lower()
    results = []
    for path in _doc_paths():
        text = path.read_text(encoding="utf-8")
        idx = text.lower().find(q)
        if idx != -1:
            start = max(0, idx - 40)
            end = min(len(text), idx + len(q) + 40)
            snippet = text[start:end].replace("\n", " ").strip()
            results.append({"name": path.stem, "snippet": f"...{snippet}..."})
    return results


if __name__ == "__main__":
    mcp.run()
```

File: data/orders.json

```
{
  "NP-100245": {
    "order_id": "NP-100245",
    "status": "shipped",
    "email": "dana.lee@example.com",
    "items": ["TENT-2P-RX", "BAG-20F-DN"],
    "tracking": "1Z999AA10123456784",
    "ordered_at": "2025-05-28",
    "delivered_at": null
  },
  "NP-100190": {
    "order_id": "NP-100190",
    "status": "delivered",
    "email": "sam.rivera@example.com",
    "items": ["BOOT-GTX-M", "FILT-PMP"],
    "tracking": "1Z999AA10198765432",
    "ordered_at": "2025-05-30",
    "delivered_at": "2025-06-04"
  },
  "NP-100201": {
    "order_id": "NP-100201",
    "status": "processing",
    "email": "dana.lee@example.com",
```

```
    "items": ["STOVE-CANN", "FUEL-230"],
    "tracking": null,
    "ordered_at": "2025-06-10",
    "delivered_at": null
  },
  "NP-100177": {
    "order_id": "NP-100177",
    "status": "delivered",
    "email": "sam.rivera@example.com",
    "items": ["JACKET-RN-L"],
    "tracking": "1Z999AA10111222333",
    "ordered_at": "2025-04-15",
    "delivered_at": "2025-04-20"
  }
}
```

File: data/docs/returns-policy.md

```
# Returns Policy

## Return window
You may return most items within **30 days** of the delivery date. The
30-day clock starts on the date the order is marked **delivered**, not the
order date. Orders that have not yet been delivered are not eligible to
start a return.

## Condition rules
- Items must be **unused** and in their **original packaging** with tags attached.
- **Footwear** (any item whose SKU begins with `BOOT-`) can only be returned
  if it is **unworn**. Worn footwear cannot be returned for hygiene reasons.
- Water filters and purifiers (SKU prefix `FILT-`) can be returned only if the
  seal is intact and the unit has not been used.
- Final-sale items are not returnable.

## How to start a return
Contact support with the order ID. Approved returns are refunded to the
original payment method within 5-7 business days of inspection.
```

File: data/docs/shipping-policy.md

```
# Shipping Policy

## Processing time
Orders are processed within 1-2 business days. You will receive a tracking
number by email once the order ships.

## Delivery estimates
- Standard shipping: 3-5 business days after dispatch.
- Expedited shipping: 1-2 business days after dispatch.

## Tracking
Every shipped order has a tracking number. An order in `processing` status
has not shipped yet and will not have a tracking number.
```

File: data/docs/warranty.md

```
# Warranty

## Coverage
NorthPeak gear carries a **1-year limited warranty** against manufacturing
```

```
defects, starting on the delivery date.

## What is covered
- Seam failures, zipper defects, and delamination under normal use.
- Stove and filter components that fail due to manufacturing faults.

## What is not covered
- Normal wear and tear, accidental damage, or misuse.
- Cosmetic wear that does not affect function.

Warranty claims are separate from returns and have no 30-day limit.
```

Appendix A — Sample codebase (starting / pre-migration state)

File: sample_codebase/src/analytics.ts

```
// Analytics helpers.
//
// logEvent(name, payload) is DEPRECATED. Do not use it in new code.
// New code should call track({ name, props }) instead.

/** @deprecated Use track({ name, props }) instead. */
export function logEvent(name: string, payload: Record<string, unknown>): void {
  console.log(`[deprecated logEvent] ${name}`, payload);
}

export function track(event: { name: string; props: Record<string, unknown> }):
void {
  console.log(`[track] ${event.name}`, event.props);
}
```

File: sample_codebase/src/notifications.ts

```
import { logEvent } from "./analytics";

export function sendOrderShipped(orderId: string, email: string): void {
  // ... send the "your order shipped" email ...
  logEvent("order_shipped_email", { orderId, email });
}

export function sendReturnApproved(orderId: string): void {
  // ... send the "return approved" email ...
  logEvent("return_approved_email", { orderId });
}
```

File: sample_codebase/src/orders.ts

```
import { logEvent } from "./analytics";

export function markDelivered(orderId: string): void {
  logEvent("order_delivered", { orderId });
}

export function cancelOrder(orderId: string, reason: string): void {
  logEvent("order_cancelled", { orderId, reason });
}
```

File: sample_codebase/tests/notifications.test.ts

```
import { sendOrderShipped, sendReturnApproved } from "../src/notifications";

describe("notifications", () => {
  it("sends an order-shipped email without throwing", () => {
    expect(() => sendOrderShipped("NP-100245",
"dana.lee@example.com")).not.toThrow();
  });

  it("sends a return-approved email without throwing", () => {
    expect(() => sendReturnApproved("NP-100190")).not.toThrow();
  });
});
```

File: sample_codebase/tests/orders.test.ts

```
import { markDelivered, cancelOrder } from "../src/orders";

describe("orders", () => {
  it("marks an order delivered without throwing", () => {
    expect(() => markDelivered("NP-100190")).not.toThrow();
  });

  it("cancels an order without throwing", () => {
    expect(() => cancelOrder("NP-100201", "customer request")).not.toThrow();
  });
});
```